

Full Index for the Subject — “Data Structures (CST205)”

Based on your official WBSCTVESD syllabus and AICTE reference book (NB Venkateswarlu).

UNIT 1 – Introduction to Data Structures (3 Hours)

1. **Basic Terminology**
2. **Classification of Data Structures**
3. **Operations on Data Structures**

UNIT 2 – Linear Data Structures (12 Hours)

Stacks

1. Introduction to Stacks
2. Array Representation of Stacks
3. Operations on a Stack (Push, Pop, Peek)
4. Applications of Stacks
 - Infix to Postfix Conversion
 - Evaluation of Postfix Expression

Queues

1. Introduction to Queues
2. Array Representation of Queues
3. Operations on Queues (Insert, Delete)
4. Types of Queues
 - Dequeue
 - Circular Queue
5. Applications: Round Robin Scheduling

Recursion

1. Concept of Recursion
2. Examples:
 - GCD (Greatest Common Divisor)
 - Tower of Hanoi

UNIT 3 – Linked Lists (15 Hours)

1. Singly Linked List
 - Representation in Memory
 - Operations: Insertion, Deletion (beginning, middle, end)
2. Circular Linked List
 - Representation
 - Operations
3. Doubly Linked List
 - Representation
 - Operations
4. Circular Doubly Linked List
 - Representation
 - Operations
5. Linked List Implementation of
 - Stack
 - Queue

UNIT 4 – Non-Linear Data Structures (15 Hours)

Trees

1. Basic Terminology
2. Definition and Concept of Binary Trees
3. Representation (Array and Linked List)
4. Operations: Insertion, Deletion, Traversal (Inorder, Preorder, Postorder)
5. Types of Binary Trees
6. B-Tree
7. AVL Tree

Graphs

1. Graph Terminologies

2. Representation

- Set Representation
- Linked Representation
- Matrix Representation

3. Graph Traversals

- BFS (Breadth First Search)
- DFS (Depth First Search)

✔ UNIT 1 – Topic 1: Basic Terminology

◆ 1. Data

Data means **raw facts and figures** — like numbers, characters, or symbols — that have no meaning until processed.

Example: 85, Sagar, A+, 1001

◆ 2. Information

Information is **processed data** that has meaning and can be used for decision-making.

Example: “Sagar scored 85 marks in Data Structures.”

◆ 3. Data Structure

A **data structure** is a way to **store and organize data** in a computer so that it can be used efficiently.

Example: Arrays, Linked Lists, Stacks, Queues, Trees, Graphs.

◆ 4. Element / Data Item

A single unit of data in a data structure.

Example: In array $A[5] = \{2, 4, 6, 8, 10\}$, each number is an element.

◆ 5. Group Item / Record

A collection of related data items.

Example:

$Student = \{RollNo, Name, Marks\}$

◆ 6. Field

Each individual item inside a record.

Example: “Name” or “RollNo” inside a student record.

◆ 7. Node

A fundamental unit in **linked structures** (like Linked Lists, Trees, Graphs) containing **data + pointer/link** to next element.

Example:

[Data | Next]

◆ 8. Key

A special field used to **uniquely identify a record**.

Example: Roll number in a student record.

◆ 9. Primitive Data Type

Basic data types provided by a programming language.

Example: `int`, `float`, `char`, `double`.

◆ 10. Non-Primitive Data Type

Derived or user-defined data types made using primitive ones.

Example: `Array`, `Structure`, `Union`, `Linked List`.

◆ 11. Memory Representation

Every data item is stored in **binary form** (0s and 1s) in **contiguous or linked memory blocks**, depending on the data structure type.

◆ 12. Algorithm

A **step-by-step procedure** for solving a problem in a finite number of steps.

Example: Algorithm to find the largest element in an array.

Summary Tip 🧠:

Think of *data structures* as the **containers** and *algorithms* as the **recipes** for processing what's inside those containers.

✔ UNIT 1 – Topic 2: Classification of Data Structures

◆ 1. Linear Data Structures

Data elements are arranged **sequentially (one after another)**.

Examples:

- **Array**
 - **Stack**
 - **Queue**
 - **Linked List**
-

◆ 2. Non-Linear Data Structures

Data elements are **connected in a hierarchical or network manner**, not sequentially.

Examples:

- **Tree**
 - **Graph**
-

◆ 3. Static Data Structures

Fixed size, memory is allocated at **compile-time**.

Example: **Array**

◆ 4. Dynamic Data Structures

Size can **change at runtime**, memory allocated **dynamically**.

Examples: **Linked List, Stack, Queue**

◆ 5. Homogeneous Data Structures

All elements are of the **same data type**.

Example: **Array**

◆ 6. Non-Homogeneous Data Structures

Elements can be of **different data types**.
Examples: **Structure**, **Class**

☑ UNIT 1 – Topic 3: Operations on Data Structures

◆ 1. Traversing

Visiting each element of a data structure **exactly once** to perform some operation (like display or search).
▣ *Example*: Printing all elements of an array.

◆ 2. Insertion

Adding a **new element** to the data structure.
▣ *Example*: Inserting a number at a specific index in an array or node in a linked list.

◆ 3. Deletion

Removing an **existing element** from a data structure.
▣ *Example*: Deleting a node from a linked list.

◆ 4. Searching

Finding the **location of a particular element** within the structure.
▣ *Example*: Linear Search, Binary Search.

◆ 5. Sorting

Arranging data elements in a specific **order** (ascending or descending).
▣ *Example*: Bubble Sort, Selection Sort, Quick Sort.

◆ 6. Merging

Combining two similar data structures into **one single structure**.
▣ *Example*: Merging two sorted arrays.

◆ 7. Updating

Changing the **value of an existing element**.

▣ *Example:* Updating a student's marks in a record.

✓ Summary Table:

Operation	Description	Example
Traversal	Visit each element	Print all array elements
Insertion	Add element	Add node to linked list
Deletion	Remove element	Delete from queue
Searching	Find element	Linear/Binary search
Sorting	Arrange data	Bubble Sort
Merging	Combine	Merge two arrays
Updating	Modify	Change marks in record

✓ UNIT 2 – Topic 1: Stacks (Introduction & Representation)

◆ 1. Definition

A **Stack** is a **linear data structure** that follows the rule:

👉 **LIFO** – *Last In, First Out*

That means the **last element inserted** is the **first to be removed**.

▣ **Example:**

Think of a stack of plates — you add (push) plates on top and remove (pop) from the top.

◆ 2. Basic Operations

Operation	Description
Push(x)	Insert an element x into the stack.
Pop()	Remove the top element from the stack.
Peek() / Top()	View the top element without removing it.
isEmpty()	Check if the stack is empty.
isFull()	Check if the stack is full (for array implementation).

◆ 3. Representation

Stacks can be represented in two ways:

1. Array Representation

- Fixed size stack.
- Uses an array and a variable **TOP** to track the top element.

Example:

```
#define MAX 5
int stack[MAX];
int TOP = -1;
```

2. Linked List Representation

- Dynamic size stack.
 - Each node contains data and pointer to next node (used later in linked list topic).
-

◆ 4. Stack Status

Condition	Meaning
-----------	---------

TOP == -1	Stack is Empty
------------------	----------------

TOP == MAX - 1	Stack is Full
-----------------------	---------------

◆ 5. Real-life Examples

- Undo/Redo in text editors
 - Browser back/forward buttons
 - Function calls (recursion)
 - Expression evaluation (postfix, prefix)
-

☑ UNIT 2 – Topic 2: Operations on Stacks (Push, Pop, Peek)

◆ 1. Push Operation

Purpose: Add an element on **top of the stack**.

Algorithm (Array-based):

1. Check if $TOP == MAX - 1 \rightarrow$ Stack Overflow.
2. Increment TOP by 1.
3. Place the new element at $stack[TOP]$.

C Example:

```
void push(int x) {  
    if(TOP == MAX-1) {  
        printf("Stack Overflow\n");  
    } else {  
        TOP++;  
        stack[TOP] = x;  
    }  
}
```

◆ 2. Pop Operation

Purpose: Remove the **top element** from the stack.

Algorithm:

1. Check if $TOP == -1 \rightarrow$ Stack Underflow.
2. Store $stack[TOP]$ in a variable (optional).
3. Decrement TOP by 1.

C Example:

```
int pop() {
    if(TOP == -1) {
        printf("Stack Underflow\n");
        return -1;
    } else {
        return stack[TOP--];
    }
}
```

◆ 3. Peek / Top Operation

Purpose: View the top element **without removing** it.

C Example:

```
int peek() {
    if(TOP == -1) {
        printf("Stack is empty\n");
        return -1;
    } else {
        return stack[TOP];
    }
}
```

◆ 4. Stack Status Checks

- **isEmpty()** → **TOP == -1**
- **isFull()** → **TOP == MAX-1**

✓ **Key Point:** Push adds, Pop removes, Peek shows. Always follow **LIFO order**.

✔ UNIT 2 – Topic 3: Realization of Stacks Using Arrays

◆ 1. Concept

- Stack can be **implemented using an array**.
 - Use a **TOP** variable to track the **index of the last inserted element**.
 - Array-based stack has **fixed size**.
-

◆ 2. Structure

```
#define MAX 5
int stack[MAX];
int TOP = -1;
```

- **TOP = -1** → Stack is empty
 - **TOP = MAX-1** → Stack is full
-

◆ 3. Push Operation

```
void push(int x) {
    if(TOP == MAX-1) printf("Stack Overflow\n");
    else stack[++TOP] = x;
}
```

◆ 4. Pop Operation

```
int pop() {
    if(TOP == -1) {
        printf("Stack Underflow\n");
        return -1;
    } else return stack[TOP--];
}
```

◆ 5. Peek Operation

```
int peek() {
    if(TOP == -1) return -1; // empty stack
    else return stack[TOP];
}
```

◆ 6. Traversal / Display

```
void display() {  
    if(TOP == -1) printf("Stack is empty\n");  
    else {  
        for(int i = TOP; i >= 0; i--) {  
            printf("%d ", stack[i]);  
        }  
        printf("\n");  
    }  
}
```

◆ 7. Advantages

- Easy to implement
- Access to TOP element is **O(1)**

◆ Limitations

- Fixed size → cannot exceed array limit
 - Memory may be wasted if stack is small
-

☑ UNIT 2 – Topic 4: Applications of Stacks

◆ 1. Expression Evaluation

- Stacks are used to **evaluate arithmetic expressions**.
- Commonly used for **Postfix (Reverse Polish Notation) evaluation**.

Algorithm for Postfix Evaluation:

1. Read expression from left to right.
 2. If operand → push onto stack.
 3. If operator → pop two elements, apply operator, push result back.
 4. At the end, the stack contains the **final result**.
-

◆ 2. Infix to Postfix Conversion

- Converts **human-readable infix expression** $(A + B) * C \rightarrow$ **Postfix** $AB+C^*$.
- Uses **operator precedence** and **stack** for operators.

Algorithm:

1. Initialize empty stack for operators.
 2. Scan infix expression:
 - If operand $\rightarrow$ output it
 - If $(\rightarrow$ push to stack
 - If $) \rightarrow$ pop and output until $($ is found
 - If operator $\rightarrow$ pop operators with higher/equal precedence from stack, then push current operator
 3. Pop remaining operators from stack
-

◆ 3. Real-life Examples

- Undo/Redo in editors
 - Function calls (Call Stack)
 - Syntax parsing in compilers
-

☑ UNIT 2 – Topic 5: Queues (Introduction & Operations)

◆ 1. Definition

A **Queue** is a **linear data structure** that follows the rule:

👉 **FIFO** – *First In, First Out*

The **first element inserted** is the **first to be removed**.

📌 **Example:** People waiting in a ticket line.

◆ 2. Basic Operations

Operation	Description
Enqueue(x)	Insert element x at the rear of the queue.
Dequeue()	Remove element from the front of the queue.
Front() / Peek()	View the front element without removing it.
isEmpty()	Check if queue is empty.
isFull()	Check if queue is full (for array implementation).

◆ 3. Array Representation

```
#define MAX 5
int queue[MAX];
int FRONT = -1, REAR = -1;
```

Status:

- **FRONT == -1** → Empty
 - **REAR == MAX-1** → Full (linear queue)
-

◆ 4. Linear Queue Operations

Enqueue:

```
void enqueue(int x){
    if(REAR == MAX-1) printf("Queue Overflow\n");
    else {
        if(FRONT == -1) FRONT = 0;
        queue[++REAR] = x;
    }
}
```

Dequeue:

```
int dequeue(){
    if(FRONT == -1) { printf("Queue Underflow\n"); return -1; }
    int val = queue[FRONT];
    if(FRONT == REAR) FRONT = REAR = -1; // Queue becomes empty
    else FRONT++;
    return val;
}
```

}

✔ UNIT 2 – Topic 6: Types of Queues & Applications

◆ 1. Simple / Linear Queue

- Elements inserted at **rear** and removed from **front**.
- Limitation: After multiple deletions, **space at the front remains unused**.

◆ 2. Dequeue (Double-Ended Queue)

- Elements can be **inserted or deleted at both ends**.
- Types:
 1. **Input-restricted dequeue** → insertion at one end, deletion at both ends
 2. **Output-restricted dequeue** → deletion at one end, insertion at both ends

◆ 3. Circular Queue

- Overcomes linear queue limitation by connecting **rear to front** (circularly).
- Allows **efficient memory usage**.
- Condition for full: $(\text{REAR} + 1) \% \text{MAX} == \text{FRONT}$
- Condition for empty: $\text{FRONT} == -1$

◆ 4. Applications

- **CPU Scheduling** → Round Robin Algorithm
 - **Printer Queue** → Jobs processed in order
 - **Data Buffers** → IO Buffers in devices
 - **Traffic Control** → Vehicles passing in sequence
-

✓ UNIT 2 – Topic 7: Recursion (Concept + Examples)

◆ 1. Definition

Recursion is a process where a **function calls itself** to solve a problem.

- Must have a **base condition** to stop recursion.

▣ Formula:

Recursive Function = Base Case + Recursive Call

◆ 2. Advantages

- Simplifies **complex problems** into smaller sub-problems.
 - Useful for problems naturally **divisible into similar sub-problems**.
-

◆ 3. Examples

a) GCD (Greatest Common Divisor)

- Using Euclid's algorithm:

```
int gcd(int a, int b){  
    if(b == 0) return a; // Base case  
    return gcd(b, a % b); // Recursive call  
}
```

b) Tower of Hanoi

- Move **n** disks from source to destination using auxiliary rod.

Algorithm:

1. Move **n-1** disks from source → auxiliary
2. Move 1 disk from source → destination
3. Move **n-1** disks from auxiliary → destination

C Example:

```
void towerOfHanoi(int n, char source, char dest, char aux){
    if(n == 1) {
        printf("Move disk 1 from %c to %c\n", source, dest);
        return;
    }
    towerOfHanoi(n-1, source, aux, dest);
    printf("Move disk %d from %c to %c\n", n, source, dest);
    towerOfHanoi(n-1, aux, dest, source);
}
```

✔ UNIT 3 – Topic 1: Singly Linked List (Representation & Operations)

◆ 1. Definition

A **Singly Linked List (SLL)** is a linear data structure where:

- Each node contains **data** and a **pointer to the next node**.
- The last node points to **NULL**.

▣ Node Structure in C:

```
struct Node {
    int data;
    struct Node* next;
};
```

◆ 2. Advantages over Arrays

- Dynamic size → can grow or shrink at runtime
 - Efficient insertion/deletion (no need to shift elements)
 - Memory used only as needed
-

◆ 3. Memory Representation

- Each node is stored **anywhere in memory**
- **head** pointer points to the **first node**

head → [data|next] → [data|next] → ... → NULL

◆ 4. Basic Operations

Operation	Description
Insertion	Add a node at beginning, middle, or end
Deletion	Remove a node from beginning, middle, or end
Traversal	Visit all nodes to display data
Search	Find a node containing a given value

◆ 5. Insertion Examples

a) At Beginning:

```
newNode->next = head;  
head = newNode;
```

b) At End:

```
temp = head;  
while(temp->next != NULL) temp = temp->next;  
temp->next = newNode;  
newNode->next = NULL;
```

c) At Position **pos**:

- Traverse to **pos-1** node → adjust **next** pointers
-

◆ 1. Definition

A **Circular Singly Linked List** is a list where:

- The **last node points back to the first node** (**head**).
- Can be traversed **continuously** from any node.

▤ **Node Structure:** Same as singly linked list.

◆ 2. Representation

head → [data|next] → [data|next] → ... → [data|next] → head

◆ 3. Advantages

- Efficient circular traversal
- Useful for **round-robin scheduling** or **buffered systems**

◆ 4. Basic Operations

Operation	How it works
Insertion at Beginning	Point new node's next to head , update last node's next to new node, update head
Insertion at End	Point last node's next to new node, new node's next to head
Deletion at Beginning	Update head to head->next , last node points to new head
Deletion at End	Traverse to second-last node, update its next to head , free last node
Traversal	Start at head , move using next until back to head

◆ 1. Definition

A **Doubly Linked List** is a list where each node contains:

1. **Data**
2. **Pointer to next node** (*next*)
3. **Pointer to previous node** (*prev*)
4. Can be traversed **forward and backward**.

▣ Node Structure in C:

```
struct Node {  
    int data;  
    struct Node* next;  
    struct Node* prev;  
};
```

◆ 2. Advantages

- Bidirectional traversal
- Easy insertion/deletion at both ends or middle

◆ 3. Basic Operations

Operation	How it works
Insertion at Beginning	Update new node's <i>next</i> to head, head's <i>prev</i> to new node, update head
Insertion at End	Traverse to last node, set last node's <i>next</i> to new node, new node's <i>prev</i> to last node
Insertion at Position	Adjust <i>prev</i> and <i>next</i> pointers of adjacent nodes
Deletion at Beginning	Move head to <i>head->next</i> , set new head's <i>prev</i> to NULL
Deletion at End	Traverse to last node, set second-last node's <i>next</i> to NULL, free last node
Traversal	Forward: head → next → ... → NULL; Backward: last → prev → ... → head

◆ 1. Definition

A **Circular Doubly Linked List** is a doubly linked list where:

- The last node's **next** points to the head
- The head's **prev** points to the last node
- Allows **continuous forward and backward traversal**

▣ **Node Structure:** Same as DLL.

◆ 2. Representation

head.prev → last node

last.next → head

◆ 3. Advantages

- Efficient **bidirectional circular traversal**
- Useful for **round-robin applications** and **buffered data structures**

◆ 4. Basic Operations

Operation	How it works
Insertion at Beginning	Update new node's next to head, prev to last node; update last node's next and head's prev
Insertion at End	Set last node's next to new node, new node's prev to last node, new node's next to head, head's prev to new node
Deletion at Beginning	Update head to head->next , adjust last node's next and new head's prev
Deletion at End	Adjust second-last node's next to head, head's prev to second-last node, free last node
Traversal	Forward: head → next → ... → head; Backward: head → prev → ... → head

◆ 1. Stack using Linked List

- **Top** pointer points to the **head of the linked list**.
- **Push**: Insert new node at **beginning**.
- **Pop**: Remove node from **beginning**.

▣ C Example (Push):

```
struct Node* top = NULL;
```

```
void push(int x){
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = top;
    top = newNode;
}
```

▣ C Example (Pop):

```
int pop(){
    if(top == NULL) return -1; // stack empty
    struct Node* temp = top;
    int val = top->data;
    top = top->next;
    free(temp);
    return val;
}
```

◆ 2. Queue using Linked List

- Maintain **front** and **rear** pointers.
- **Enqueue**: Insert node at **rear**.
- **Dequeue**: Remove node from **front**.

▣ C Example (Enqueue):

```
struct Node *front = NULL, *rear = NULL;
```

```
void enqueue(int x){
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = x; newNode->next = NULL;
    if(rear == NULL) { front = rear = newNode; return; }
    rear->next = newNode;
    rear = newNode;
}
```

C Example (Dequeue):

```
int dequeue(){
    if(front == NULL) return -1; // queue empty
    struct Node* temp = front;
    int val = front->data;
    front = front->next;
    if(front == NULL) rear = NULL;
    free(temp);
    return val;
}
```

UNIT 4 – Topic 1: Trees (Basic Terminology & Binary Tree Concepts)

1. Definition of Tree

A **tree** is a **non-linear data structure** that consists of **nodes connected by edges**.

- One node is **root**
 - Other nodes are **children**
 - Nodes with no children are **leaf nodes**
-

2. Basic Terminology

Term	Meaning
Node	Basic unit containing data
Root	Topmost node of the tree
Parent	Node directly above a node
Child	Node directly below a node
Leaf Node	Node with no children
Sibling	Nodes sharing the same parent
Degree of Node	Number of children a node has

Height of Node	Longest path from node to a leaf
Depth / Level	Distance from root node
Degree of Tree	Maximum number of children among all nodes

◆ 3. Binary Tree

- Each node has **at most 2 children**:
 - **Left child**
 - **Right child**
-

◆ 4. Types of Binary Trees

1. **Full Binary Tree** → Every node has 0 or 2 children
 2. **Complete Binary Tree** → All levels except last are full, last level filled left to right
 3. **Perfect Binary Tree** → All internal nodes have 2 children, all leaves at same level
 4. **Skewed Tree** → All nodes have only one child (left or right)
-

✓ UNIT 4 – Topic 2: Binary Tree Representation (Array & Linked List)

◆ 1. Array Representation

- Store **binary tree elements in an array** sequentially.
- Root at index **0** (or **1** depending on implementation).
- For node at index i :
 - **Left child** → $2*i + 1$
 - **Right child** → $2*i + 2$
 - **Parent** → $(i-1)/2$

Advantages:

- Simple for **complete binary trees**

Disadvantages:

- Wastes memory for sparse trees
-

◆ 2. Linked List Representation

- Each node contains:
 - **Data**
 - **Pointer to left child**
 - **Pointer to right child**

▣ C Structure Example:

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};
```

Advantages:

- Memory-efficient for **sparse trees**
- Easy to add/delete nodes

Disadvantages:

- Uses extra memory for pointers
-

☑ UNIT 4 – Topic 3: Binary Tree Operations (Insertion, Deletion, Traversals)

◆ 1. Insertion

- Add a new node at the **first available position** (level-wise) to maintain tree structure.
 - In **BST (Binary Search Tree)**:
 - Left child < Parent
 - Right child > Parent
-

◆ 2. Deletion

- **Leaf node**: Simply remove it.
 - **Node with one child**: Replace node with its child.
 - **Node with two children**: Replace with **inorder predecessor** or **successor**, then delete that node.
-

◆ 3. Traversals

a) Depth-First Traversals (DFS)

1. Inorder (L → Root → R)

- Left subtree → Root → Right subtree
- Used in BST → returns **sorted order**

2. Preorder (Root → L → R)

- Root → Left → Right
- Useful for **copying tree**

3. Postorder (L → R → Root)

- Left → Right → Root

- Useful for **deleting tree**

b) Breadth-First Traversal (BFS / Level Order)

- Visit nodes **level by level** from root
 - Implemented using a **queue**
-

✓ UNIT 4 – Topic 4: Types of Binary Trees, B-Tree, AVL Tree

◆ 1. Types of Binary Trees

Type	Description
Full Binary Tree	Every node has 0 or 2 children
Complete Binary Tree	All levels except last are full; last level filled left to right
Perfect Binary Tree	Full and all leaves at same level
Skewed Tree	All nodes have only one child (left or right)
Balanced Binary Tree	Height difference of left & right subtree ≤ 1 for every node

◆ 2. B-Tree

- **Balanced search tree** used in **databases & file systems**
- Each node can have **multiple keys** and **children**
- Maintains **sorted data** and allows **fast insertion, deletion, search**

- Properties:
 - All leaves at same level
 - Minimum and maximum number of keys per node depends on order m
-

◆ 3. AVL Tree

- **Self-balancing Binary Search Tree**
- For every node:
 - $\text{Height}(\text{left subtree}) - \text{Height}(\text{right subtree}) \leq 1$
- If imbalance occurs → perform **rotations** (single or double) to balance

Rotations:

1. **Left Rotation (LL)**
 2. **Right Rotation (RR)**
 3. **Left-Right Rotation (LR)**
 4. **Right-Left Rotation (RL)**
-

☑ UNIT 4 – Topic 5: Graphs (Terminologies, Representations, BFS & DFS)

◆ 1. Definition

A **Graph** is a **non-linear data structure** consisting of:

- **Vertices (nodes)**
 - **Edges (connections between vertices)**
-

◆ 2. Terminologies

Term	Meaning
Vertex / Node	Fundamental unit containing data
Edge	Connection between two vertices
Degree of Vertex	Number of edges connected to a vertex
Path	Sequence of edges connecting vertices
Cycle	Path that starts and ends at same vertex
Directed Graph (Digraph)	Edges have direction
Undirected Graph	Edges have no direction
Weighted Graph	Edges have weights
Unweighted Graph	Edges have no weights

◆ 3. Graph Representations

1. Adjacency Matrix

- 2D array; `matrix[i][j] = 1` if edge exists
- Easy to implement

- Takes **$O(V^2)$** space

2. Adjacency List

- Each vertex has a list of **connected vertices**
- Space-efficient for **sparse graphs**

3. Set Representation

- Edges stored as **pairs** (u, v)

◆ 4. Graph Traversals

a) BFS (Breadth-First Search)

- Visit vertices **level by level**
- Implemented using a **queue**
- Useful for **shortest path in unweighted graphs**

b) DFS (Depth-First Search)

- Visit vertices **deeply before backtracking**
 - Implemented using **stack or recursion**
 - Useful for **topological sorting, cycle detection**
-